

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' QA-VALIDACION (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: Página Web Productora 2 (SCRUM-20)

1. CONTEXTO

Se requiere una página web de presentación corporativa para una productora audiovisual. El stack está explícitamente restringido a HTML5 + CSS + JS puro (sin frameworks de backend, sin bases de datos, sin Node.js). El sitio es 100% estático con 4 secciones: Home, Servicios, Quiénes Somos y Contacto. El formulario de contacto enviará datos vía servicio externo (EmailJS o Formspree) dado que no existe backend permitido. El requerimiento menciona integración CMS, pero dado que el stack es HTML/CSS/JS puro, esto queda FUERA del alcance del MVP actual (YAGNI aplicado).

2. DECISIÓN ARQUITECTÓNICA

****Arquitectura elegida:** Sitio Estático de Página Única (Single-Page con scroll suave) — Monolito HTML/CSS/JS**

Estructura de UN solo archivo HTML principal (`index.html`) con secciones `

****Por qué NO se eligió una solución más compleja:****

- 'L ****NO React/Vue/Angular****: El requerimiento dice explícitamente 'HTML5, CSS y JS solamente'. Agregar un framework JS añadiría un proceso de build innecesario y viola la restricción del cliente.
- 'L ****NO arquitectura multi-página (MPA con archivos separados por sección)****: Con 4 secciones de contenido estático, una SPA con scroll es más performante (0 requests adicionales de navegación), más simple de mantener y cumple todos los criterios de aceptación. El menú de navegación usa anclas (`href='#servicios'`) con `scroll-behavior: smooth`.
- 'L ****NO backend propio para el formulario****: El stack no lo permite. Se usará EmailJS (SDK JS del lado cliente) que cumple el criterio de 'enviar correo sin almacenar datos en BD'. Alternativa: Formspree como endpoint POST.
- 'L ****NO CMS integrado en este MVP****: El requerimiento arquitectónico menciona CMS, pero el stack HTML/CSS/JS puro lo hace imposible sin backend. Riesgo asumido conscientemente: el contenido será estático hasta que se apruebe un sprint de integración CMS (Jekyll, Hugo o headless CMS).
- 'L ****NO CDN configurada en código****: La CDN es responsabilidad de infraestructura/hosting (Cloudflare, Netlify, Vercel), no del código fuente. El código se entregará optimizado para ser desplegado en cualquiera de estas plataformas.

3. JUSTIFICACIÓN YAGNI/KISS

- ****YAGNI****: No se implementa i18n (español/inglés) en este MVP porque el requerimiento funcional no incluye un mecanismo de selección de idioma ni traducciones aprobadas. Se deja estructura preparada con atributo `lang='es'` y comentarios para futura expansión.
- ****KISS****: Un solo `index.html` + 3 archivos CSS + 2 archivos JS es suficiente para cumplir los 6 escenarios de aceptación. Cada archivo tiene responsabilidad única y clara.
- ****Seguridad XSS****: Toda manipulación del DOM en el formulario usa `textContent` en lugar de `innerHTML`. La validación es client-side con sanitización de inputs. EmailJS maneja el envío de forma segura.
- ****Imágenes****: 100% placeholders via `https://placeholder.co` y `https://picsum.photos`. Ningún archivo de imagen local.

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (BAJO)**: EmailJS tiene límite de 200 emails/mes en plan gratuito. Si el volumen supera esto, se requiere plan de pago o migración a backend. Mitigación: documentado en README.
- **Riesgo 2 (MEDIO)**: Sin backend, no hay protección CSRF real en el formulario. EmailJS mitiga esto con tokens de servicio del lado cliente. La protección SQL Injection es irrelevante (no hay BD). XSS mitigado por sanitización JS.
- **Riesgo 3 (BAJO)**: La integración CMS queda pendiente. El contenido estático deberá ser editado directamente en HTML hasta el próximo sprint. Riesgo aceptado por el cliente según scope del requerimiento.
- **Riesgo 4 (BAJO)**: Las pruebas de carga (100 usuarios simultáneos) son responsabilidad del hosting estático (Netlify/Vercel/GitHub Pages manejan esto nativamente). El código no introduce cuellos de botella.

2. MÉTRICAS DE ITERACIÓN (QA)

- productora-web/index.html | Estado: APROBADO | Iteraciones: 1
- productora-web/js/navigation.js | Estado: APROBADO | Iteraciones: 1
- productora-web/js/contact.js | Estado: APROBADO | Iteraciones: 1
- productora-web/css/variables.css | Estado: APROBADO | Iteraciones: 1
- productora-web/css/main.css | Estado: WARNING (Forzado) | Iteraciones: 3
- productora-web/css/responsive.css | Estado: APROBADO | Iteraciones: 2
- productora-web/README.md | Estado: APROBADO | Iteraciones: 2

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 91,726

Tokens de salida (output): 25,591

Total tokens: 117,317

Horas-hombre estimadas: ~8.9 horas

Modelo utilizado: claude-sonnet-4-6

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"4,166 input | !"1,621 output | 1 llamadas

MICROPLANIFICACION-DISEÑO: !"17,510 input | !"10,499 output | 7 llamadas

FRONTEND-DESARROLLO: !"31,511 input | !"10,879 output | 11 llamadas

QA-VALIDACION: !"38,539 input | !"2,592 output | 11 llamadas